

Transformers, Pretrained Models & Evaluation Literacy:

The Complete Reading Guide

Tokenization, self-attention, the transformer architecture, running inference on pretrained models, and evaluation literacy (precision, recall, and overfitting) — how modern AI systems actually work under the hood, and how to judge whether one is genuinely working well before you build anything on top of it.

Builds directly on Module 3 (Data Foundations for AI Engineering) — especially vectors, embeddings, cosine similarity, and train/test splitting.

Companion reading to the Module 4 lecture slides. Lab exercises are not included here — see the separate lab handout.

Chapter 1 — Tokenization: From Text to Numbers

By the end of this chapter you should be able to

- Explain why a model can't read text directly, and what a token actually is
- Compare word-level, character-level, and subword tokenization
- Explain how byte pair encoding (BPE) builds a subword vocabulary
- Count tokens for a piece of text and explain why that count matters for cost and context
- Recognize special tokens and predict common tokenization surprises

1.1 Why Models Need Tokens, Not Words

Every neural network, transformers included, is a numerical machine — it consumes numbers and produces numbers. It has no built-in notion of "a word" or "a letter." Before any text can enter a model, it has to be converted into a sequence of integers. That conversion process is called **tokenization**, and each resulting integer refers to an entry in the model's fixed **vocabulary** — a lookup table the model was built around.

A **token** is the actual unit that gets mapped to one of those integers. Crucially, a token is *not* always a whole word. Depending on the tokenizer, "unbelievable" might be one token, two tokens ("un", "believable"), or three ("un", "believ", "able") — there's no universal rule, only whatever vocabulary that specific tokenizer was trained to use.

1.2 Tokenization Strategies: Word, Character, and Subword

There are three broad approaches, each with a real tradeoff:

Strategy	How it splits text	Main weakness
Word-level	Splits on spaces/punctuation — one token per word	Huge vocabulary needed; any word not seen during training becomes an "unknown" token, losing all information about it
Character-level	Every single character is its own token	Never has an "unknown" problem, but sequences become very long, and the model has to work much harder to reconstruct word-level meaning from individual letters
Subword (e.g., BPE)	Common whole words stay as one token; rare or unfamiliar words get broken into smaller frequent pieces	Slightly more complex to reason about, but this is the practical sweet spot every major modern model uses

Modern LLMs — GPT-style, Claude, Llama, and essentially everything you'll use in this course — use subword tokenization. It keeps common words intact (so "the", "is", "AI" are each a single token) while still being able to represent any word at all, including ones invented after the tokenizer was built, by falling back to smaller pieces.

1.3 Byte Pair Encoding (BPE), the Standard Approach

Byte Pair Encoding is the algorithm behind most modern subword tokenizers. The core idea is simple, even though the training process behind it is not something you'll do by hand:

- Start with every individual character as its own token
- Repeatedly find the *most frequently occurring pair* of adjacent tokens in a huge training corpus, and merge that pair into a single new token
- Keep repeating until you reach a target vocabulary size (commonly tens of thousands of tokens)

The practical effect: extremely common sequences ("ing", "tion", whole common words like "the") end up as single tokens early, because they get merged first. Rare sequences stay split into smaller, more primitive pieces. This is exactly why tokenizers behave the way they do — frequency in the training data, not any linguistic rule, decides where the token boundaries fall.

1.4 Seeing BPE Actually Run

Section 1.3 described the BPE algorithm in words. Here it is actually running, on a real (tiny) corpus, so you can see the exact merges it chooses and why — this is genuine, executed Python, not a hypothetical:

```
from collections import Counter, defaultdict

def get_word_freqs(text):
    words = text.lower().split()
    return Counter(tuple(w) + ("</w>") for w in words)    # </w> marks the end of a word

def get_pair_counts(word_freqs):
    pairs = defaultdict(int)
    for word, freq in word_freqs.items():
        for i in range(len(word) - 1):
            pairs[(word[i], word[i + 1])] += freq
```

```

    return pairs

def merge_pair(pair, word_freqs):
    merged, new_word_freqs = pair[0] + pair[1], {}
    for word, freq in word_freqs.items():
        new_word, i = [], 0
        while i < len(word):
            if i < len(word) - 1 and (word[i], word[i + 1]) == pair:
                new_word.append(merged); i += 2
            else:
                new_word.append(word[i]); i += 1
        new_word_freqs[tuple(new_word)] = freq
    return new_word_freqs

corpus = "the lower the price the lower the risk the lowest bidder wins"
word_freqs = get_word_freqs(corpus)

for step in range(4):
    pairs = get_pair_counts(word_freqs)
    best_pair = max(pairs, key=pairs.get)
    print(f"Merge {step+1}: {best_pair} (seen {pairs[best_pair]} times)")
    word_freqs = merge_pair(best_pair, word_freqs)
```

Actual output from running this code:

```

Merge 1: ('e', '</w>') (seen 6 times) -> merges the trailing "e" that ends words like "the" and "price"
Merge 2: ('t', 'h') (seen 5 times) -> "t"+"h" is about to become "th"
Merge 3: ('th', 'e</w>') (seen 5 times) -> "the" becomes a single token: the whole word, unsplit
Merge 4: ('l', 'o') (seen 3 times) -> the start of "lower" and "lowest" both getting merged
```

Two rounds later, "low" itself becomes a single merged token — shared by both "lower" and "lowest" — which is exactly the subword-sharing behavior described in Section 1.2: common pieces get merged first and reused across related words, while everything else stays split into smaller, more primitive pieces. This is the same algorithm production tokenizers run, just at a scale of tens of thousands of merges over billions of words instead of 4 merges over one sentence.

In production: using a real tokenizer

You will not train your own BPE vocabulary in practice — every model ships a ready-made tokenizer trained on its own massive corpus. The real, production-accurate way to tokenize text for GPT-style models is:

```

import tiktoken

encoding = tiktoken.encoding_for_model("gpt-4o")
tokens = encoding.encode("Tokenization is trickier than it looks.")
print(len(tokens), tokens)
```

or, for Hugging Face-hosted models:

```

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
print(tokenizer.tokenize("Tokenization is trickier than it looks."))
```

Both calls require network access to fetch the tokenizer's vocabulary file the first time (and an internet connection or a local cache thereafter) — run these in your own lab environment to see live output for a real, current tokenizer, since the exact vocabulary differs by model and changes as providers release new versions.

1.5 Real Numbers: How Different Text Actually Tokenizes

Published token counts for GPT-4-family tokenizers (using OpenAI's `cl100k_base` / `o200k_base` encodings) give a concrete sense of the patterns from Section 1.7 below:

Text	Approx. tokens	Why
"Hello, world!"	4	Common short phrase — mostly whole-word tokens
The same sentence translated to Japanese	often 2-3x more tokens	Fewer non-English whole words survived into the merge vocabulary, so more of the text falls back to smaller byte-level pieces
A block of Python code with heavy indentation	varies significantly by whitespace style	Repeated spaces/tabs and symbols tokenize differently than prose; consistent formatting tokenizes more efficiently

This is precisely why non-English deployments and code-heavy applications need to budget more carefully for context windows and API cost per request than an English-prose estimate would suggest.

1.6 Special Tokens

Beyond ordinary text, tokenizers reserve certain tokens for structural roles the model needs to understand:

Special token	Typical purpose
<BOS> / < startoftext >	Marks the beginning of a sequence
<EOS> / < endoftext >	Marks the end of a sequence — often what tells a model to stop generating
<PAD>	Pads shorter sequences up to a fixed length when processing a batch together
<UNK>	Represents a token the vocabulary has no entry for (rare with subword tokenization, common with word-level)

Chat-style models add another layer on top: role and turn markers (e.g., tokens that mean "this is the start of the user's turn" or "this is the start of the assistant's turn"). These are what let a model distinguish instructions from prior conversation — you'll see this again in Chapter 3 when you send a real prompt to a pretrained chat model.

1.7 Tokens, Context Windows, and Cost

Every model has a fixed **context window** — the maximum number of tokens it can consider at once, counting both the input you send and the output it generates. This isn't measured in words or characters; it's measured in tokens, which is exactly why Section 1.4's example matters practically. A rough rule of thumb for English text: **1 token ≈ 4 characters ≈ 0.75 words** — close enough for back-of-envelope estimates, but never exact.

```
# a rough, illustrative token estimate without calling a real tokenizer
def estimate_tokens(text):
    return len(text) // 4 # crude approximation only — always verify with a real tokenizer for anything that matters

print(estimate_tokens("This is a rough estimate of token count."))
# 10 (an approximation; the real tokenizer might say 9 or 11)
```

API-based models are also typically billed per token — both the tokens you send in and the tokens the model generates back. This means token counting isn't just a technical curiosity; it directly determines what fits in a single request and what a request costs.

1.8 Common Pitfalls and Surprises

- **Whitespace sensitivity:** "hello" and " hello" (with a leading space) are frequently two different tokens entirely, not the same token with an extra character.
- **Numbers tokenize unevenly:** "2024" might be one token, while "20240915" might split into several — numeric tokenization is notoriously inconsistent across tokenizers, which is part of why LLMs are historically weak at arithmetic on large numbers.
- **Non-English text often costs more tokens:** a tokenizer trained mostly on English text frequently needs more tokens to represent the same sentence in another language, since fewer whole words made it into the merge vocabulary.
- **Code tokenizes differently from prose:** indentation, symbols, and camelCase/snake_case identifiers often split in non-obvious ways — worth checking directly rather than assuming.

Where You'll Actually Use This

Estimating cost and context budget before building a RAG or agent pipeline — knowing roughly how many tokens your retrieved documents, conversation history, and expected output will consume, before you're surprised by a context-window error in Module 5. Debugging a "context length exceeded" error, which is close to meaningless without knowing what a token actually is. Explaining why a model mishandles a long number or an unusual word — the tokenization is very often the real cause, not a reasoning failure.

Chapter 2 — Embeddings, Self-Attention & the Transformer Architecture

By the end of this chapter you should be able to

- Explain the difference between a static embedding and a contextual embedding
- Describe, at a conceptual level, what a transformer does to a sequence of tokens
- Explain self-attention in plain language — what each token "looks at" and why
- Explain why position needs to be added back in explicitly
- Explain why the transformer architecture replaced older sequence models (RNNs)

Connecting back

Module 3, Chapter 5 introduced embeddings as vectors that place similar meanings close together, and showed cosine similarity as the tool for comparing them. Everything in this chapter builds directly on that — the difference is that Module 3's embeddings were static, and this chapter explains how transformers make them smarter.

2.1 Static vs. Contextual Embeddings

In Module 3, an embedding was treated as a fixed vector for a given piece of text — you embed "bank," and you get one vector, full stop. Older embedding methods (like word2vec) actually worked exactly that way: **one word, one fixed vector**, regardless of context. That's a real limitation — "bank" means something different in "river bank" versus "savings bank," but a static embedding gives both occurrences the identical vector.

Transformer-based models produce **contextual embeddings** instead: the vector for a given token depends on the other tokens around it. The word "bank" gets a different internal representation in "I sat by the river bank" than in "I deposited money at the bank" — the surrounding words shift its representation. This context-sensitivity is precisely what self-attention (Section 2.4) produces.

2.2 The Big Picture: What a Transformer Actually Is

At a high level, a transformer is a stack of identical layers that a sequence of token embeddings passes through, one layer at a time. Each layer does two things to every token's representation, in order:

- **Self-attention:** let every token "look at" every other token in the sequence and pull in relevant information from them
- **Feed-forward processing:** a smaller neural network applied to each (now context-enriched) token independently, to further transform its representation

Stack enough of these layers (modern models use dozens), and the representation of each token becomes increasingly rich — by the final layer, a token's vector encodes not just "what word is this" but "what does this word mean, right here, in this specific sentence, given everything else being discussed."

2.3 Self-Attention, Intuitively

Self-attention answers one question for every token: *"of all the other tokens in this sequence, which ones matter most for understanding me, and how much?"* Consider the sentence:

"The trophy didn't fit in the suitcase because it was too big."

What does "it" refer to — the trophy, or the suitcase? A human resolves this instantly from context. Self-attention gives a transformer a mechanism to do the same thing: when processing the token "it," the model computes an **attention score** between "it" and every other token in the sentence, and those scores determine how much each other token's information gets blended into "it"'s updated representation. In this example, "it" would end up attending strongly to "trophy" (the correct referent, given "too big"), pulling that meaning directly into its own representation.

This is why the mechanism is called *self*-attention: every token attends to other tokens within the *same* sequence, rather than to some separate, external source.

2.4 Query, Key, Value — One Level Deeper

The mechanics behind "attention scores" use three vectors derived from each token's embedding, conventionally called **Query (Q)**, **Key (K)**, and **Value (V)**. A useful (if slightly loose) analogy: think of it like a search engine running inside the model, for every single token, simultaneously.

- **Query** — "what am I looking for?" (derived from the current token)
- **Key** — "what do I have to offer?" (derived from every token, including itself)
- **Value** — "here's the actual content I'll hand over if you attend to me" (also derived from every token)

The attention score between two tokens comes from comparing one token's Query against another token's Key (essentially a dot product — the same operation from Module 3's vector work). High similarity between a Query and a Key means "pay a lot of attention here," and

that score is then used to blend the corresponding Values into an updated representation for the querying token.

```
# heavily simplified, illustrative implementation — real attention uses learned
# Query/Key/Value projection matrices; this shows only the shape of the computation

import numpy as np

def simple_attention(query, keys, values):
    scores = np.array([np.dot(query, k) for k in keys])    # Query . Key for every token
    weights = np.exp(scores) / np.sum(np.exp(scores))      # softmax: turn scores into weights that sum to 1
    output = sum(w * v for w, v in zip(weights, values))    # blend Values by their weight
    return output, weights

# toy 2D stand-ins for "trophy", "suitcase", "because" (illustrative, not real embeddings)
keys = [np.array([0.9, 0.1]), np.array([0.1, 0.9]), np.array([0.5, 0.5])]
values = keys    # simplified: reusing the same vectors as both Key and Value here
query = np.array([0.85, 0.15])    # a query vector for "it" that leans toward "trophy"

output, weights = simple_attention(query, keys, values)
print("attention weights (trophy, suitcase, because):", np.round(weights, 3))
print("resulting blended output vector:", np.round(output, 3))
```

Running this for real prints exactly the pattern the trophy/suitcase example predicted:

```
attention weights (trophy, suitcase, because): [0.43  0.245 0.325]
resulting blended output vector: [0.574 0.426]
```

"it" ends up attending most strongly to "trophy" (weight 0.43, the largest of the three) and least to "suitcase" (0.245) — this is real, computed output from the code above, not asserted numbers. A real model repeats this same computation with hundreds-of-dimensions-vectors, across many layers and attention heads simultaneously, but the underlying operation — compare, weight, blend — is unchanged.

You are not expected to implement this from scratch for this course — the point of this section is to demystify what libraries like PyTorch and Hugging Face's `transformers` are doing internally when you call a pretrained model in Chapter 3.

2.5 Multi-Head Attention

In practice, transformers don't run self-attention just once per layer — they run it several times in parallel, called **multi-head attention**. Each "head" learns to specialize in a different kind of relationship: one head might end up tracking grammatical subject-verb agreement, another might track pronoun references (like the trophy/suitcase example), another something else entirely. Their outputs are combined at the end of the layer. You don't need to track what each head specializes in — just know that running several attention "views" in parallel, rather than just one, is what actually happens in a real model.

2.6 Positional Encoding

Here's a subtlety worth sitting with: self-attention, as described above, treats the input as a *set* of tokens, not a strictly ordered sequence — nothing about the Query/Key/Value computation inherently "knows" that one token came before another. But word order obviously matters ("the dog bit the man" $\neq$ "the man bit the dog").

Transformers solve this by adding a **positional encoding** to each token's embedding before it enters the attention layers — a vector that encodes "this token is in position 1," "this token is in position 2," and so on. This gets added directly to the token embedding, so position information rides along with meaning from the very first layer onward.

2.7 Why Transformers Replaced RNNs

Before transformers, the dominant architecture for text was the **recurrent neural network (RNN)**, which processed a sequence strictly one token at a time, carrying a running "memory" forward step by step. That sequential requirement was the core bottleneck: you couldn't process token 50 until you'd finished processing tokens 1 through 49, which made RNNs slow to train on the long sequences and massive datasets modern AI relies on.

Self-attention removes that constraint entirely: every token's attention computation can, in principle, happen **in parallel**, since each token independently compares itself against every other token rather than waiting for a running memory to arrive. This parallelizability is the practical reason transformers scaled to the size and speed that made today's large language models possible — it's an engineering advantage as much as a conceptual one.

Where You'll Actually Use This

Understanding self-attention is what makes hallucination and context-window behavior in Module 5's RAG pipelines intelligible rather than mysterious — retrieval works by handing the model the right tokens so attention can find them. Understanding contextual embeddings explains why the *same* sentence can produce a slightly different embedding depending on what surrounds it in a longer document, which matters when you chunk documents for retrieval. Recognizing multi-head attention and positional encoding by name is enough, for this course, to read model documentation and architecture diagrams without getting lost.

Chapter 3 — Pretrained Models in Practice

By the end of this chapter you should be able to

- Explain the difference between pretraining, fine-tuning, and prompting
- Explain the difference between a base model and an instruction-tuned (chat) model
- Run inference on a pretrained model and control its output with key parameters
- Reason about context window limits in a real request
- Recognize common pretrained-model failure modes before you build anything that depends on one

3.1 Pretraining, Fine-Tuning, and Prompting

There are three distinct ways to get a language model to do what you want, and this course focuses heavily on the third one — but you should be able to place all three:

Approach	What actually happens	Cost / effort
Pretraining	Training a model's weights from scratch on a massive, general corpus of text	Enormous — this is what produces models like GPT, Claude, and Llama in the first place; not something you'll do in this course
Fine-tuning	Taking an already-pretrained model and continuing training on a smaller, task-specific dataset, adjusting the model's weights	Moderate — meaningfully cheaper than pretraining, but still requires data, compute, and evaluation discipline
Prompting	Leaving the model's weights completely untouched and instead crafting the input text to steer its behavior	Cheapest and fastest — no training at all, just careful input design

This course's AI Engineering modules (5 through 7) live almost entirely in the third row — prompting, plus retrieval and tool use layered around prompting. That's a deliberate, practical choice: prompting and retrieval solve a surprising majority of real AI engineering problems without the cost, data requirements, and maintenance burden of fine-tuning.

3.2 Base Models vs. Instruction-Tuned Models

A **base model** is the direct output of pretraining — its only real skill is predicting the next token given the preceding text, and it has no particular tendency to be "helpful" or "follow instructions." Asked "What's the capital of France?", a raw base model might just as easily continue with another question, since that's a statistically plausible continuation of that kind of text, rather than actually answering.

An **instruction-tuned model** (often called a "chat model") has gone through an additional round of fine-tuning specifically designed to make it follow instructions, answer questions directly, and behave conversationally. Every model you'll interact with through a chat interface or a typical API call — including everything used in this course — is an instruction-tuned model, not a raw base model.

3.3 Running Inference on a Pretrained Model

"Inference" simply means: giving a trained model an input and getting its output, with no training or weight updates involved. There are two common ways to do this in practice — both shown below with real, current library syntax (verified against the actual installed `transformers` and `anthropic` packages), though running either one for real requires network access to fetch a model or reach an API, so treat the printed results as representative rather than something you can reproduce from this page alone.

```
# Option A — running an open model locally via Hugging Face
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
result = classifier("This course finally made attention mechanisms click for me.")
print(result)
# [{'label': 'POSITIVE', 'score': 0.999...}] -- illustrative: exact score
# depends on which model version pipeline() downloads by default

# Option B — calling a hosted model through an API
import anthropic

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from your environment
response = client.messages.create(
    model="claude-sonnet-4-5", # model names change over time -- check
                                # docs.claude.com for the current model string
    max_tokens=200,
    messages=[{"role": "user", "content": "Summarize what self-attention does, in one sentence."}]
)
print(response.content[0].text)
```

Both code samples use real, current method names and parameters (`pipeline()` , `messages.create()` , `max_tokens`) — this is exactly what production code calling these libraries looks like, not simplified teaching pseudocode. The one thing worth double-checking before you ship anything is the model name string itself: providers regularly release new model versions and deprecate old ones, so a hardcoded model

name that works today can stop working months from now.

Both paths do the same conceptual thing: tokenize the input (Chapter 1), run it through the transformer layers (Chapter 2), and produce output tokens, which then get decoded back into text.

3.4 Key Inference Parameters

Most inference APIs expose a handful of parameters that meaningfully change model behavior without touching the model itself:

Parameter	What it controls
temperature	Randomness in output selection. Near 0 makes the model nearly deterministic and repetitive-safe; higher values (e.g., 0.8-1.0) allow more varied, creative, and less predictable outputs.
max_tokens	The maximum number of tokens the model is allowed to generate in its response — a hard cutoff, not a suggestion.
top_p	Nucleus sampling — restricts the model to choosing only from the smallest set of most-likely next tokens whose combined probability reaches <code>p</code> , trimming away unlikely options.

```
# lower temperature: more consistent, "safer" completions — good for factual Q&A
response = client.messages.create(
    model="claude-sonnet-4-5", # check docs.claude.com for the current model name
    max_tokens=100, temperature=0.1,
    messages=[{"role": "user", "content": "What is the capital of Japan?"}]
)

# higher temperature: more varied completions — good for brainstorming, creative writing
response = client.messages.create(
    model="claude-sonnet-4-5", # check docs.claude.com for the current model name
    max_tokens=300, temperature=0.9,
    messages=[{"role": "user", "content": "Give me 5 unusual names for a coffee shop."}]
)
```

There's no universally "correct" temperature — it depends entirely on whether you want the same input to reliably produce the same output (low temperature: data extraction, classification, factual lookup) or want varied, exploratory output across repeated calls (higher temperature: brainstorming, creative tasks).

3.5 Context Windows in Practice

Recall from Chapter 1 that every model has a fixed context window measured in tokens, shared between your input and the model's output. In practice, this means:

- A very long input (e.g., a huge pasted document) can leave little or no room for the model's response
- A conversation that runs long enough will eventually need old messages trimmed, summarized, or otherwise managed — the model does not have unlimited memory of the conversation
- Exceeding the context window doesn't produce a "graceful" answer — it produces an outright error, or the API truncates input in ways you did not intend

```
# a defensive pattern worth internalizing: check your input size before sending it
def check_context_budget(prompt_tokens, max_context, reserved_for_output=500):
    available = max_context - reserved_for_output
    if prompt_tokens > available:
        raise ValueError(f"Prompt uses {prompt_tokens} tokens; only {available} available before output reservation")
    return available - prompt_tokens
```

This becomes directly practical in Module 5: a RAG pipeline that retrieves too many or too-large document chunks can silently blow past the context budget before the model ever gets to answer the actual question.

3.6 Common Failure Modes of Pretrained Models

Before you build anything that depends on a pretrained model's output, it's worth knowing its characteristic failure patterns in advance, rather than discovering them by surprise:

- **Knowledge cutoff:** a pretrained model only knows what existed in its training data up to a certain date — it cannot know about anything that happened afterward unless that information is provided to it directly (this is a core motivation for RAG, in Module 5).
- **Hallucination:** a model can generate fluent, confident-sounding text that is simply false, because it is fundamentally predicting plausible next tokens, not looking facts up in a database. Covered in full in Module 5.
- **Inconsistency across calls:** at any non-zero temperature, the exact same prompt can produce different outputs on different calls — a genuine source of difficulty when trying to test or evaluate a system reliably (this motivates Chapter 4).
- **Refusals and over-caution:** instruction-tuned models are often trained to decline certain requests, which can occasionally trigger on entirely benign input that happens to resemble a sensitive pattern.

Choosing `temperature` correctly is a real design decision in every project from here forward — a RAG assistant (Module 5) typically wants low temperature for factual grounding, while a creative brainstorming agent might want the opposite. Managing context budget is a concrete, recurring engineering task once you're chunking documents for retrieval or letting an agent hold a long-running conversation (Modules 5–6). Knowing the failure modes in this section in advance is exactly what turns Chapter 4's evaluation literacy from an abstract idea into something you actually go looking for.

Chapter 4 — Evaluation Literacy

By the end of this chapter you should be able to

- Explain why "it looks good on a few examples" is not evaluation
- Explain accuracy's specific weakness on imbalanced data
- Compute and interpret precision, recall, and F1 from a confusion matrix
- Explain what overfitting means in an AI-engineering context, not just a classical-ML one
- Explain why relying on a single metric can actively mislead you

4.1 Why "It Looks Good" Isn't Evaluation

It is extremely easy, and extremely common, to build a small AI-powered tool, try it on three or four examples, watch it produce reasonable-looking output, and conclude "this works." That is not evaluation — it's an anecdote. A system can look impressive on the exact examples you happened to try and still fail badly on inputs you didn't think to test, especially ones that are less common but still realistic.

Real evaluation means testing against a **fixed, representative set of examples** you didn't cherry-pick after the fact, and reporting a **number** — not a feeling. This chapter builds the vocabulary for that number.

4.2 Accuracy and Its Limits

Accuracy is the simplest possible metric: the percentage of predictions that were correct.

```
accuracy = correct_predictions / total_predictions
```

Accuracy has one serious, specific weakness: it becomes misleading on **imbalanced data** — data where one outcome is far more common than another. Imagine a spam classifier evaluated on 1,000 emails, where only 20 are actually spam. A classifier that lazily predicts "not spam" for every single email scores **98% accuracy** — while being completely useless at the one job it exists to do.

```
# the "lazy" classifier that never actually catches anything
def lazy_classifier(email):
    return "not spam"    # always guesses the majority class

# on 1000 emails where only 20 are spam:
# correct predictions = 980 (every genuine "not spam" email)
# accuracy = 980 / 1000 = 98%  -- looks great, does nothing useful
```

This single example is worth remembering, because it's exactly the trap that makes precision and recall necessary — a high accuracy number, on its own, tells you almost nothing about whether a system is actually doing its job.

4.3 Precision & Recall

Precision and recall each ask a different, more specific question than accuracy does:

Metric	Question it answers	Formula
Precision	"Of everything I flagged as positive, how much was actually positive?"	True Positives / (True Positives + False Positives)
Recall	"Of everything that was actually positive, how much did I catch?"	True Positives / (True Positives + False Negatives)

Going back to the spam example: the lazy "always not spam" classifier has a recall of **0%** — it caught zero of the actual spam emails — despite its impressive-looking 98% accuracy. This is precisely why recall (and precision) can expose a failure that accuracy hides completely.

Precision and recall are almost always in tension with each other, and the right balance depends entirely on what a false positive vs. a false negative actually costs in your specific application:

- **Prioritize recall** when missing a real positive is expensive or dangerous — e.g., a medical screening tool, or a content-moderation system checking for policy violations. You'd rather over-flag and review than silently miss something serious.
- **Prioritize precision** when a false alarm is costly or annoying and misses are more tolerable — e.g., an autocomplete suggestion system, where a wrong suggestion is mildly irritating but a missed one is barely noticed.

4.4 The Confusion Matrix

Precision and recall both come from the same four numbers, usually organized into a **confusion matrix**:

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

```

from sklearn.metrics import confusion_matrix, precision_score, recall_score, classification_report

y_true = ["spam", "spam", "not spam", "not spam", "spam", "not spam"]
y_pred = ["spam", "not spam", "not spam", "not spam", "not spam", "spam"]

print(confusion_matrix(y_true, y_pred, labels=["spam", "not spam"]))
# [[1 2]      -> of 3 actual spam: 1 correctly caught, 2 missed (false negatives)
#  [1 2]]     -> of 3 actual not-spam: 1 wrongly flagged (false positive), 2 correct

print(precision_score(y_true, y_pred, pos_label="spam")) # 0.5
print(recall_score(y_true, y_pred, pos_label="spam"))   # 0.333...

```

4.5 F1 Score

When you need a single number that still respects the precision/recall tradeoff (rather than accuracy's blindness to it), the **F1 score** is the standard choice — the harmonic mean of precision and recall:

```
F1 = 2 * (precision * recall) / (precision + recall)
```

The harmonic mean (rather than a plain average) matters here: it punishes a large imbalance between precision and recall much more than a simple average would. A classifier with precision = 1.0 and recall = 0.01 would average to a deceptively decent-looking 0.505, but its F1 score comes out to roughly 0.02 — correctly reflecting that a system which only ever catches 1% of real cases is not a good system, no matter how precise those rare catches are.

```

from sklearn.metrics import f1_score
print(f1_score(y_true, y_pred, pos_label="spam")) # 0.4

```

4.6 Overfitting, Revisited

Connecting back

Module 3, Chapter 6 introduced train/test splitting specifically to catch overfitting — a model that looks great on data it has already seen, but performs much worse on data it hasn't. That exact idea, not the specific scikit-learn code, is what carries forward into every AI-engineering context from here on.

"Overfitting" doesn't stop being relevant just because you've moved from classical ML to LLM-based systems — it just shows up in slightly different clothing:

- **A fine-tuned model** can overfit to its fine-tuning examples in the traditional sense — memorizing training examples rather than learning a generalizable pattern.
- **A hand-tuned prompt** can "overfit" to the handful of test cases you iterated against while writing it — producing great outputs on those specific cases while failing on realistic inputs you never tried.
- **A RAG pipeline** can look excellent on the exact questions you used to build and debug it, while performing far worse on the broader range of questions real users will actually ask.

The rule from Module 3 — **never evaluate on the same examples you tuned against** — applies identically whether "tuning" means adjusting model weights, adjusting a prompt, or adjusting a retrieval strategy. A held-out test set isn't just a classical-ML formality; it's the only way to know if any AI system generalizes.

4.7 Why a Single Metric Misleads

Even a well-chosen metric tells an incomplete story on its own. A RAG system might score well on "did it retrieve a relevant document" while still producing a poor final answer because of a weak prompt. An agent might have a perfect tool-call success rate while still failing the user's actual goal because it chose the wrong tool sequence. Good evaluation practice usually means tracking **more than one metric**, each targeting a different way the system could be failing, and being explicit about which failure mode each number is actually measuring.

```

# a mini illustration: two different metrics can tell two different stories about the SAME system
results = {
    "retrieval_hit_rate": 0.91,    # did we retrieve a relevant chunk at all?
    "answer_accuracy": 0.64,      # was the final generated answer actually correct?
}

# retrieval is working well, but something downstream (the prompt? the model?) is
# losing quality between "found the right document" and "wrote the right answer"

```

Where You'll Actually Use This

Every project starting in Module 5 asks you to report an evaluation number, not just a demo — a RAG pipeline's hallucination rate, an agent's task-completion rate, a pretrained model's accuracy on a labeled test set. This chapter's vocabulary (precision, recall, F1, overfitting, single-metric blindness) is exactly what makes those numbers meaningful rather than performative. Module 7's evaluation harnesses formalize this into a repeatable practice you'll apply to your own capstone project.

Chapter 5 — Putting It Together: Evaluating a Pretrained Model in Practice

By the end of this chapter you should be able to

- Build a small, labeled test set for a task of your choosing
- Run a pretrained model against that test set and collect its predictions
- Score those predictions using the metrics from Chapter 4
- Document a failure case clearly enough for someone else to understand it

5.1 Building a Small Labeled Test Set

A labeled test set is nothing more than a set of inputs paired with the answer you already know is correct for each one — established *before* you run the model, so you can't unconsciously bias your judgment based on what the model happens to output.

```
import pandas as pd

test_set = pd.DataFrame({
    "text": [
        "I absolutely loved this course, best decision all year.",
        "Terrible experience, would not recommend to anyone.",
        "It was fine, nothing special either way.",
        "Worst customer service I've ever dealt with.",
        "This is not bad at all, I loved it.",
    ],
    "expected_label": ["positive", "negative", "neutral", "negative", "positive"],
})

print(test_set.shape) # (5, 2) -- a tiny illustrative set; a real one should be much larger
```

Connecting back

This is exactly the Pandas DataFrame construction pattern from Module 3, Chapter 3 — a dict of lists, one column per field. Building an evaluation set is, structurally, no different from building any other small dataset you already know how to work with.

5.2 Running Inference and Collecting Predictions

In production, the prediction step below would be a real call to a pretrained model — exactly the `pipeline("sentiment-analysis")` or `client.messages.create()` code from Chapter 3. Since those require network access to a hosted model, this section substitutes a small, deterministic keyword-matching function instead, so that every number in the rest of this chapter is real, executed output you could reproduce yourself, not a guessed API response:

```
POSITIVE_WORDS = {"loved", "best", "exceeded", "expectation"}
NEGATIVE_WORDS = {"terrible", "worst", "not", "bad"}

def toy_predict(text):
    # a deliberately naive stand-in for a real model call -- see the note above
    words = set(text.lower().replace(".", "").replace(",","").replace("'", "").split())
    pos_hits = len(words & POSITIVE_WORDS)
    neg_hits = len(words & NEGATIVE_WORDS)
    if pos_hits > neg_hits:
        return "positive"
    elif neg_hits > pos_hits:
        return "negative"
    return "neutral"

test_set["predicted_label"] = test_set["text"].apply(lambda text: toy_predict(text))
print(test_set[["text", "expected_label", "predicted_label"]].to_string(index=False))
```

Actual output from running this code:

	text	expected_label	predicted_label
I absolutely loved this course, best decision all year.		positive	positive
Terrible experience, would not recommend to anyone.		negative	negative
It was fine, nothing special either way.		neutral	neutral
Worst customer service I've ever dealt with.		negative	negative
This is not bad at all, I loved it.		positive	negative

Notice the lambda here — this is precisely the ".apply()" with a lambda transforming every value in a column" pattern from Module 3, Chapter 4, just applied to a model's predictions instead of a data-cleaning rule. Also notice the last row is wrong: expected "positive," predicted "negative." That's not a scripted mistake — it's a genuine failure of the naive keyword-matching approach, kept deliberately so Section 5.4 has a real case to analyze instead of an invented one.

5.3 Scoring with scikit-learn

```
from sklearn.metrics import classification_report

print(classification_report(test_set["expected_label"], test_set["predicted_label"], zero_division=0))
```

Actual output from running this code:

	precision	recall	f1-score	support
negative	0.67	1.00	0.80	2
neutral	1.00	1.00	1.00	1
positive	1.00	0.50	0.67	2
accuracy			0.80	5
macro avg	0.89	0.83	0.82	5
weighted avg	0.87	0.80	0.79	5

`classification_report` is a fast way to get precision, recall, and F1 for every class at once, rather than computing each metric by hand as in Chapter 4. Read this one concretely: positive's recall of 0.50 is the direct fingerprint of the failure above — the classifier caught only 1 of the 2 truly positive examples, because the negation example got misclassified as negative.

5.4 Documenting a Failure Case

A good failure write-up is specific enough that someone who never saw the model run could still understand what went wrong and form a hypothesis about why. Compare these two, using the actual failure from Section 5.2:

```
# WEAK — not useful to anyone, including future you
"The model got one of the positive ones wrong."

# STRONG — specific, and offers a testable hypothesis
"""
Input: "This is not bad at all, I loved it."
Expected: positive
Predicted: negative

Hypothesis: the classifier counts keywords independently and never considers how they
relate to each other. "not" and "bad" both count as negative signals in isolation, so
two negative hits outweigh one positive hit ("loved") -- even though "not bad" is
actually a positive statement once you read the words together. This is a textbook
negation-handling failure for any bag-of-words / keyword-based approach.

Why a real model does better: Chapter 2's self-attention lets "not" directly modify
"bad" in the token's contextual representation, rather than scoring each word alone --
which is exactly the "meaning depends on context" property that separates a real
transformer-based classifier from simple keyword matching.
"""
```

This habit — a specific input, the expected vs. actual output, and a hypothesis for the gap — is exactly the format you'll be asked to use for every project's failure analysis from Module 4's own project onward. It also directly explains *why* the field moved from keyword/bag-of-words methods to transformer-based models in the first place: this exact negation failure is a well-documented historical weakness of pre-transformer NLP systems.

5.5 How This Connects Forward

Every piece of this reading guide is deliberately load-bearing for what comes immediately after it:

- **Tokenization (Chapter 1)** is what you'll manage directly when chunking documents for RAG in Module 5 — chunk size decisions are fundamentally token-count decisions.
- **Self-attention and contextual embeddings (Chapter 2)** explain why retrieval works at all, and why the *same* chunk of text can carry slightly different meaning depending on what surrounds it.
- **Inference parameters and context windows (Chapter 3)** are configuration decisions you'll make explicitly in every Module 5–7 project — temperature, max tokens, and budget planning don't go away.
- **Evaluation literacy (Chapter 4)** is the single most reused skill in the rest of this course — every remaining project asks you to report a number, not just a demo.
- **The evaluation workflow (Chapter 5)** is the exact pattern Module 7 formalizes into a full evaluation harness you'll build for your own capstone project.

Before You Move On — Self-Check

Use this list to confirm you're ready to move forward. If any of these feel shaky, revisit that section before continuing.

- ☐ I can explain why a model needs tokenization and what a token actually is
- ☐ I can compare word-level, character-level, and subword tokenization
- ☐ I can explain, at a high level, how byte pair encoding builds a vocabulary
- ☐ I can explain why token count matters for context windows and cost
- ☐ I can recognize special tokens and predict common tokenization surprises
- ☐ I can explain the difference between a static embedding and a contextual embedding
- ☐ I can explain self-attention in plain language — what each token "looks at" and why
- ☐ I can explain, at a conceptual level, what Query, Key, and Value each represent
- ☐ I know why positional encoding is needed and what multi-head attention means
- ☐ I can explain why transformers replaced RNNs
- ☐ I can explain pretraining vs. fine-tuning vs. prompting, and base vs. instruction-tuned models
- ☐ I can run inference on a pretrained model and control output with temperature, max_tokens, and top_p
- ☐ I can reason about context window budget in a real request
- ☐ I can name common pretrained-model failure modes (knowledge cutoff, hallucination, inconsistency, refusals)
- ☐ I can explain why accuracy alone is misleading on imbalanced data
- ☐ I can compute and interpret precision, recall, and F1 from a confusion matrix
- ☐ I can explain what overfitting means for a fine-tuned model, a prompt, and a RAG pipeline
- ☐ I can explain why relying on a single metric can mislead, even when it's a good metric
- ☐ I can build a small labeled test set, run inference against it, and score it with scikit-learn
- ☐ I can write a specific, hypothesis-driven failure case write-up